Decorator Design Pattern

Agenda

1. Motivation
2. Naive Solution
3. Problems
4. Decorator Pattern
5. Class Diagram
6. Implementation
7. Advantages and Disadvantages
8. Other Patterns
9. Facade Pattern
10. Relationships with Other Design Patterns
11. Real-world Applications
12. Mini Quiz

1. A Real-world Problem

Imagine your team is building an order management software for a coffee shop chain (e.g., The Coffee House).
Available coffee:

- Dark Roast
- House Blend
- Espresso

Customers rarely order plain coffee.
They often request additional condiments:

- Milk
- Sugar
- Caramel

Each topping changes

- Price
- Description

2. Naive Solution - Inheritance

The first idea is using inheritance.

Example:

- Beverage
- DarkRoast
- DarkRoastWithMilk
- DarkRoastWithMilkAndSugar
- DarkRoastWithMilkAndSugarAndWhip
- EspressoWithWhip
- HouseBlendWithMilk
- ...

2. Naive Solution - Inheritance

The first idea is using inheritance.

Example:

- Beverage
- DarkRoast
- DarkRoastWithMilk
- DarkRoastWithMilkAndSugar
- DarkRoastWithMilkAndSugarAndWhip
- EspressoWithWhip
- HouseBlendWithMilk
- ...

Problem

Every combination requires a new class.

Naive Solution - Boolean Flags

Another approach is storing every topping inside the Beverage class.

```
class Beverage {

private:

    bool hasMilk;
    bool hasSugar;
    bool hasWhip;
    bool hasCaramel;

public:
    ...
};
```

Naive Solution - Boolean Flags

Another approach is storing every topping inside the Beverage class.

```
class Beverage {

private:

    bool hasMilk;
    bool hasSugar;
    bool hasWhip;
    bool hasCaramel;

public:
    ...
};
```

Everything is controlled by boolean variables.

3. Problems with the Naive Solution

Class Explosion

4 coffee types $\times$ many toppings $=$ dozens (or hundreds) of subclasses.

3. Problems with the Naive Solution

Class Explosion

4 coffee types $\times$ many toppings $=$ dozens (or hundreds) of subclasses.

Hard Maintenance

Changing the price of milk requires modifying many classes.

3. Problems with the Naive Solution

Class Explosion

4 coffee types $\times$ many toppings $=$ dozens (or hundreds) of subclasses.

Hard Maintenance

Changing the price of milk requires modifying many classes.

Violates Open/Closed Principle

Adding a new topping (e.g. Boba)
requires modifying Beverage.

3. Problems with the Naive Solution

Class Explosion

4 coffee types $\times$ many toppings $=$ dozens (or hundreds) of subclasses.

Hard Maintenance

Changing the price of milk requires modifying many classes.

Violates Open/Closed Principle

Adding a new topping (e.g. Boba)
requires modifying Beverage.

No Runtime Flexibility

Cannot naturally support

- Double Milk
- Triple Mocha

4. Decorator Pattern

Decorator is a **Structural Design Pattern**.

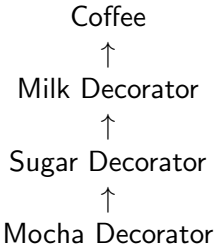
Instead of inheritance,
we dynamically attach new behaviors to an object.

Core philosophy:

Composition over Inheritance

Core Concepts

Decorator uses wrapper objects.



5. General Structure of Decorator Pattern

Main Components

- **Component**: Common interface for all objects.
- **ConcreteComponent**: The original object.
- **Decorator**: Abstract wrapper containing a Component.
- **ConcreteDecorator**: Adds new behavior while preserving the interface.

Flow:

$$\textit{Component} \leftarrow \textit{Decorator} \leftarrow \textit{Decorator} \leftarrow \textit{Decorator}$$

Every decorator is still recognized as a Component.

Coffee UML (Mermaid)

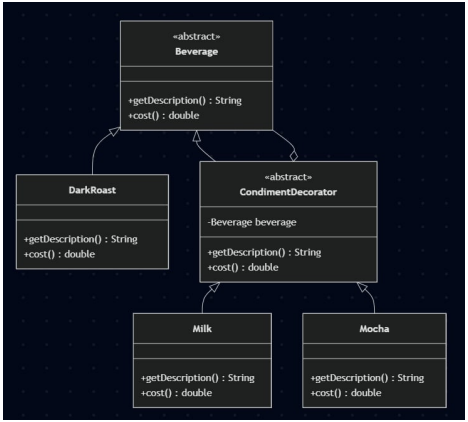


Figure: Coffee UML diagram

7. Component

```
class Beverage {
public:
    virtual string getDescription() const = 0;
    virtual double cost() const = 0;
    virtual ~Beverage() = default;
};
```

7. Component

```cpp
class Beverage {
public:
    virtual string getDescription() const = 0;
    virtual double cost() const = 0;
    virtual ~Beverage() = default;
};
```

Role

Defines the common interface. Every coffee and every topping must inherit from Beverage.

Concrete Component

```
class DarkRoast : public Beverage {
public:
    string getDescription() const override {
        return "Dark␣Roast␣Coffee";
    }

    double cost() const override {
        return 30.0;
    }
};
```

Concrete Component

```
class DarkRoast : public Beverage {
public:
    string getDescription() const override {
        return "Dark␣Roast␣Coffee";
    }

    double cost() const override {
        return 30.0;
    }
};
```

DarkRoast is the original object before decoration.

Base Decorator

```
class CondimentDecorator : public Beverage {
protected:
    Beverage* beverage;
public:
    CondimentDecorator(Beverage* bev) : beverage(bev) {}

    virtual ~CondimentDecorator() {
        delete beverage;
    }
};
```

Base Decorator

```
class CondimentDecorator : public Beverage {
protected:
    Beverage* beverage;
public:
    CondimentDecorator(Beverage* bev) : beverage(bev) {}

    virtual ~CondimentDecorator() {
        delete beverage;
    }
};
```

The decorator wraps another Beverage object.
Every decorator stores a pointer to another Beverage.

Concrete Decorator: Milk

```
class Milk : public CondimentDecorator {
public:
    Milk(Beverage* bev) : CondimentDecorator(bev) {}

    string getDescription() const override {
        return beverage->getDescription() + ", Milk";
    }

    double cost() const override {
        return beverage->cost() + 5.0;
    }
};
```

Concrete Decorator: Milk

```
class Milk : public CondimentDecorator {
public:
    Milk(Beverage* bev) : CondimentDecorator(bev) {}

    string getDescription() const override {
        return beverage->getDescription() + ", Milk";
    }

    double cost() const override {
        return beverage->cost() + 5.0;
    }
};
```

Concrete Decorator: Mocha

```
class Mocha : public CondimentDecorator {
public:
    Mocha(Beverage* bev) : CondimentDecorator(bev) {}
    string getDescription() const override { return beverage->
        getDescription() + ", Mocha";
    }

    double cost() const override {
        return beverage->cost() + 8.0;
    }
};
```

Concrete Decorator: Mocha

```
class Mocha : public CondimentDecorator {
public:
    Mocha(Beverage* bev) : CondimentDecorator(bev) {}
    string getDescription() const override { return beverage->
        getDescription() + ", Mocha";
    }

    double cost() const override {
        return beverage->cost() + 8.0;
    }
};
```

Every new decorator extends the previous Beverage.
No existing class needs modification.

Client Code

```
int main() {
    Beverage* myDrink = new DarkRoast();

    myDrink = new Milk(myDrink);
    myDrink = new Mocha(myDrink);

    cout << myDrink->getDescription() << endl;
    cout << myDrink->cost();
    delete myDrink;
}
```

How Decoration Works

The object is wrapped layer by layer.

$$DarkRoast$$

$$\Uparrow$$

$$Milk(DarkRoast)$$

$$\Uparrow$$

$$Mocha(Milk(DarkRoast))$$

How Decoration Works

The object is wrapped layer by layer.

$$DarkRoast$$

$$\Uparrow$$

$$Milk(DarkRoast)$$

$$\Uparrow$$

$$Mocha(Milk(DarkRoast))$$

Every wrapper is still a Beverage.

Execution Flow

Calling

$$myDrink->cost()$$

starts from the outermost decorator.

Execution order:

$$Mocha \rightarrow Milk \rightarrow DarkRoast$$

Then the values are returned back.

$$8 + 5 + 30 = 43$$

Program Output

Description

Dark Roast Coffee, Milk, Mocha

Total Cost

43k VND

Advantages

Runtime Flexibility

Features can be added dynamically while the program is running.

Advantages

Runtime Flexibility

Features can be added dynamically while the program is running.

Unlimited Combinations

Supports any topping combination without creating new subclasses.

Advantages

Runtime Flexibility

Features can be added dynamically while the program is running.

Unlimited Combinations

Supports any topping combination without creating new subclasses.

Reusable Components

Each decorator can be reused independently.

Advantages

Runtime Flexibility

Features can be added dynamically while the program is running.

Unlimited Combinations

Supports any topping combination without creating new subclasses.

Reusable Components

Each decorator can be reused independently.

Follows SOLID

- Single Responsibility Principle
- Open/Closed Principle

Disadvantages

Many Small Classes

Each feature requires its own decorator class.

Disadvantages

Many Small Classes

Each feature requires its own decorator class.

Harder Debugging

Many wrapper layers produce long call stacks.

Disadvantages

Many Small Classes

Each feature requires its own decorator class.

Harder Debugging

Many wrapper layers produce long call stacks.

Complex Initialization

Nested construction may become difficult.
Example:

$$new\ Mocha(new\ Milk(new\ Mocha(new\ DarkRoast())))$$

Strategy Pattern

Core Idea

Encapsulate different algorithms behind a common interface and allow them to be switched at runtime.

- One task $\rightarrow$ Multiple possible algorithms
- Each algorithm is implemented in a separate class
- The algorithm can be changed dynamically at runtime
- Avoids large `if-else` or `switch-case` blocks

Example: A graph visualizer supporting multiple pathfinding algorithms.

The Problem: Without Strategy

```
class GraphVisualizer {
public:
    void RunPathfinding(string algoType) {
        if (algoType == "Dijkstra") {
        }
        else if (algoType == "BellmanFord") {
        }
        else if (algoType == "BFS") {
        }
    }
};
```

Problems

- The class becomes very large and difficult to maintain
- Adding a new algorithm requires modifying existing code
- High risk of introducing bugs

The Solution: Strategy Pattern

```
class IPathfindingStrategy {
public:
    virtual void ExecuteAlgorithm() = 0;
    virtual ~IPathfindingStrategy() {}
};
```

Common Interface

Every pathfinding algorithm follows the same contract.

- `DijkstraStrategy`
- `BellmanFordStrategy`
- `BFSStrategy`
- `...`

Each algorithm is encapsulated in its own class. The main application does not need to know the implementation details.

Concrete Strategies

```
class DijkstraStrategy : public IPathfindingStrategy {
public:
    void ExecuteAlgorithm() override { // Dijkstra's algorithm
    }
};

class BellmanFordStrategy : public IPathfindingStrategy {
public:
    void ExecuteAlgorithm() override { // Bellman-Ford algorithm
    }
};
```

Key Idea

Different algorithms are interchangeable because they implement the same interface.

Context: Graph Visualizer

```
class GraphVisualizer {
private:
    IPathfindingStrategy* strategy;
public:
    void SetStrategy(IPathfindingStrategy* s) {
        strategy = s;
    }
    void Run() {
        if (strategy)
            strategy->ExecuteAlgorithm();
    }
};
```

Context

The `GraphVisualizer` does not care which algorithm is used. It simply executes the selected strategy.

Switching Strategies at Runtime

```
GraphVisualizer app;

// User selects Dijkstra
app.SetStrategy(new DijkstraStrategy());
app.Run();

// User switches to Bellman-Ford
app.SetStrategy(new BellmanFordStrategy());
app.Run();
```

Result

The algorithm changes, but the `GraphVisualizer` code remains unchanged.

Strategy Pattern: Summary

One task, multiple interchangeable algorithms

- Encapsulate each algorithm separately
- Use a common interface
- Switch algorithms at runtime
- Reduce complex conditional logic
- Easy to extend with new strategies

Main Benefit

Add a new algorithm without modifying the existing context.

Facade Pattern

Core Idea

Provide a simple, unified interface to a complex subsystem.

Complex System $\longrightarrow$ **Simple Interface**

- Hide unnecessary implementation details
- Reduce the complexity seen by the client
- Provide a single entry point to a subsystem

Example: Starting a computer by pressing the power button.

The Problem: A Complex Subsystem

```
class CPU {
public:
    void Freeze() {}
    void Jump(long position) {}
    void Execute() {}
};
class Memory {
public:
    void Load(long position, char* data) {}
};
class HardDrive {
public:
    char* Read(long lba, int size) {
        return "OS data";
    }
};
```

The Problem: A Complex Subsystem (cont.)

Problem

Starting a computer requires coordinating multiple components: CPU, RAM, and Hard Drive.

The Solution: Computer Facade

```
class ComputerFacade {
private:
    CPU cpu;
    Memory ram;
    HardDrive hdd;

public:
    void StartComputer() {
        cpu.Freeze();
        ram.Load(0x00, hdd.Read(0, 1024));
        cpu.Jump(0x00);
        cpu.Execute();
    }
};
```

Facade Pattern

Facade

`ComputerFacade` hides the complex startup process behind a single method: `StartComputer()`.

Client: Simple Interface

```cpp
int main() {
    ComputerFacade myPC;

    // Press the power button
    myPC.StartComputer();

    return 0;
}
```

One method call. That's it.

- The client does not interact directly with CPU, RAM, or HDD
- Internal complexity is hidden
- The client only depends on the Facade

Facade Pattern: Real-World Analogy

Press the Power Button

```
StartComputer()
```

- CPU initialization
- Loading data from the Hard Drive
- Loading data into Memory
- Starting CPU execution

The User

Only needs to press one button. They do not need to understand how the internal components work together.

Decorator, Facade, and Strategy solve different problems

Decorator	Facade	Strategy
Add Behavior	**Hide Complexity**	**Change Behavior**
Extends an object's functionality without modifying its original class.	Provides a simple interface to a complex subsystem.	Allows different algorithms to be switched at runtime.

Decorator $\rightarrow$ *What can I add?*
Facade $\rightarrow$ *How can I simplify access?*
Strategy $\rightarrow$ *How can I change the algorithm?*

Decorator, Facade, and Strategy

Pattern	Main Purpose	Key Question
Decorator	Add or extend behavior	*How can I add functionality?*
Facade	Simplify access to a complex system	*How can I make the system easier to use?*
Strategy	Switch between different algorithms	*How can I change the way a task is performed?*

The Big Picture

All three patterns help reduce coupling and improve flexibility, but they focus on different aspects of software design.

Example: A Pathfinding System

Solution

The client calls one simple method through the Facade. The Strategy selects the pathfinding algorithm, while Decorators add extra functionality.

Summary

Decorator
Adds new behavior to an object.

Facade
Hides system complexity behind a simple interface.

Strategy
Allows different algorithms to be exchanged easily.

Key Takeaway

Decorator adds behavior, Facade simplifies access, Strategy changes behavior.

Decorator is widely used in the Java Standard Library.

Example:

```
new BufferedInputStream( new FileInputStream("data.txt"))
```

Decorator is widely used in the Java Standard Library.

Example:

```
new BufferedInputStream( new FileInputStream("data.txt"))
```

Idea

FileInputStream provides basic file reading.
BufferedInputStream wraps it and adds buffering for higher performance.
The original object is not modified.

React Higher-Order Components (HOC)

A Higher-Order Component (HOC) wraps another component.

Example:

$$withAuth(ProfilePage)$$

React Higher-Order Components (HOC)

A Higher-Order Component (HOC) wraps another component.

Example:

$$withAuth(ProfilePage)$$

The wrapper may add:

- Authentication
- Logging
- Theme
- Error Handling

without changing the original component.

Backend Middleware

Frameworks such as Express.js use the same idea.

Example

$$Auth \rightarrow Logger \rightarrow Controller$$

Each middleware wraps the next one.

Backend Middleware

Frameworks such as Express.js use the same idea.

Example

$$Auth \rightarrow Logger \rightarrow Controller$$

Each middleware wraps the next one.
Typical responsibilities:

- Authentication
- Authorization
- Logging
- Request Validation

Python Decorators

Python decorators are direct implementations of the Decorator idea.

Example:

```
@login_required
def dashboard():
    ...
```

Python Decorators

Python decorators are direct implementations of the Decorator idea.

Example:

```
@login_required
def dashboard():
    ...
```

The function is wrapped before execution.
Additional behavior can be added without modifying the original function.

Flutter Widgets

Flutter also encourages composition.

Example

$$Container(Padding(Text("Hello")))$$

Flutter Widgets

Flutter also encourages composition.

Example

$$Container(Padding(Text("Hello")))$$

Each widget wraps another widget.
Instead of inheritance,
Flutter builds UI through composition.

Question 1

Decorator belongs to which Design Pattern category?

A. Creational
B. Structural
C. Behavioral
D. Architectural

Question 1

Decorator belongs to which Design Pattern category?

A. Creational
B. Structural
C. Behavioral
D. Architectural

Answer

B. Structural

Prize: Sweet Candy

Question 2

Which design principle does Decorator emphasize?

A. Inheritance over Composition
B. Composition over Inheritance
C. Tight Coupling
D. Procedural Programming

Question 2

Which design principle does Decorator emphasize?

A. Inheritance over Composition
B. Composition over Inheritance
C. Tight Coupling
D. Procedural Programming

Answer

B. Composition over Inheritance

Prize: Cake

Question 3

What is the main purpose of Decorator?

A. Clone an object
B. Restrict object creation
C. Add new behavior at Runtime
D. Simplify a subsystem

Question 3

What is the main purpose of Decorator?

A. Clone an object
B. Restrict object creation
C. Add new behavior at Runtime
D. Simplify a subsystem

Answer

C. Add new behavior at Runtime

Prize: Virtual Hug

Question 4

Which is a disadvantage of Decorator?

A. Violates OCP
B. Causes Class Explosion
C. Long call stack and many small classes
D. Cannot work in OOP languages

Question 4

Which is a disadvantage of Decorator?

A. Violates OCP
B. Causes Class Explosion
C. Long call stack and many small classes
D. Cannot work in OOP languages

Answer

C

Prize: Chocolate

Question 6

How do we implement Double Milk?

A. Create DoubleMilkDecorator
B. Add count = 2
C. Wrap Milk twice
D. Copy the code twice

Question 6

How do we implement Double Milk?

A. Create DoubleMilkDecorator
B. Add count = 2
C. Wrap Milk twice
D. Copy the code twice

Answer

C

Prize: Bubble Milk Tea

Thank You!
Questions?